

Laboratoire 06 – Portage de Linux sur la carte DE-1

Objectifs du laboratoire

Ce laboratoire permet de mettre en place l'environnement Linux pour la DE-1 SoC, ainsi que de faire un premier driver user-space afin de tester le design du laboratoire précédent.

Génération de la carte SD

Le firmware du HPS est capable de démarrer depuis une carte SD. Il s'attend à trouver le SPL sur une partition contenant l'identificateur 0xA2. Cette partition peut se trouver n'importe où. Le reste peut être partitionné n'importe comment. L'exemple suivant comprend une partition 0xA2 contenant le SPL et U-Boot. Une partition FAT32 contient le kernel Linux (zImage) et le device tree (dtb) dans une image ITB ainsi que le bitstream (rbf) pour la FPGA. La partition ext4 contient le rootfs utilisé par Linux.

```
# fdisk -lu /dev/sdX
```

...

<i>Device</i>	<i>Boot</i>	<i>Start</i>	<i>End</i>	<i>Sectors</i>	<i>Id</i>	<i>System</i>
/dev/sdX1		1	262144	262144	c	W95 FAT32
/dev/sdX2		262145	263429	1285	a2	Unknown
/dev/sdX3		263430	15564799	15301370	83	Linux

Nous vous fournissons une image de carte SD. Ceci vous fournira une base de travail stable et vous épargnera la création des partitions avec fdisk, la compilation de U-Boot et la création du rootfs avec Buildroot. Cette image utilise la version 6.6 de Linux, nous allons recompiler et mettre à jour une version plus récente du kernel.

- Récupérer l'image SD **de1soc.img.gz** sur le repo Git des laboratoires
- Décompresser l'image
- Flasher la carte à l'aide du script **deploy_img.sh** fournis sur git
 - `./deploy_img.sh -d sdX -t de1soc -i <img_path>`
 - **ATTENTION : N'ECRASEZ PAS LE DISQUE DUR PAR ERREUR!**
Ce script fait un dd sur le device donné à l'aide de -d
- Une fois flasher, démarrer la DE1-SoC avec la carte SD
- Utilisez picocom et vérifiez que votre carte démarre correctement jusqu'au login.
- L'image fournie contient une configuration SSH vous permettant de vous y connecter avec la commande suivante: `ssh root@192.168.0.2`. Bien sûr cela requiert que vous configuriez votre carte réseau afin qu'elle se trouve dans le même réseau (192.168.0.X/24), si qui sera également utile pour transmettre des fichiers sur la carte.

Compilation de Linux et du device tree

Le kernel fournis sur la carte est en version 6.6, nous souhaitons mettre à jour vers la version 6.18. Les FPGA-SoC sont supportés dans Linux mainline, contrairement à avant où Altera maintenait sa propre version pour ses FPGA-SoC. Vous pouvez trouver les sources sur Github : <https://github.com/torvalds/linux>. Ces sources permettront également de compiler des modules Linux pour les prochains laboratoires.

- Télécharger les sources, directement l'archive à l'aide de l'interface GitHub ou à l'aide de git et les options suivantes `--depth 1 --branch v6.18` afin de limiter le temps de téléchargement.
- Trouver la configuration pour les SoC FPGA et compiler le noyau avec.
 - Une toolchain ARM (gcc v11.3.1) est présente sur les VMs dans `/opt` ou disponible [ici](#).

Afin que Linux puisse trouver correctement le rootfs, il faut lui indiquer la partition à utiliser via les *bootargs*. Il y a plusieurs possibilités pour ça, pour les labos, nous allons passer par le device tree.

Dans les sources de Linux, il n'y a pas de device tree spécifique à la DE-1, il faut donc en créer un nouveau à partir d'une carte similaire. La DE-0 a un device tree dans les sources du kernel, nous allons l'utiliser comme point de départ.

- Trouvez le device tree (dts) de la DE-0 et copiez-le sous un nouveau nom.
- Editez-le et analysez-le. Regardez également le contenu du dtsi inclus.
- Enlevez les nœuds `gpio0`, `gpio1`, `gpio2` que nous n'utiliserons pas. Le nœud `i2c0` est spécifique à la DE-0 et doit être enlevé également.
- Dans les *bootargs* ajoutez `"root=/dev/mmcbk0p3 rw rootwait"` juste avant `"earlyprintk"`
- Modifiez le *Makefile* du dossier contenant les DTS pour `socfpga` afin de compiler le nouveau device tree.
- Compilez le device tree

Il faut maintenant créer une image ITB avec le nouveau kernel et device tree.

- Récupérer le fichier *de1soc.its* sur Git et modifier le pour inclure correctement la *zImage* du kernel et le *.dtb* du device tree.
- Créer l'image *itb* à l'aide de *mkimage*
 - `mkimage -f de1soc.its de1soc.itb`
 - *mkimage* est disponible sur la VM, sinon il faut installer la paquet Ubuntu *u-boot-tools* (ou équivalent en fonction de la distribution). Il est aussi possible de le compiler à partir des sources de U-Boot (voir [ici](#))
- Copier le fichier *de1soc.itb* sur la carte SD dans la partition *BOOT*

Test du bridge HPS <-> FPGA

Pour tester le bridge `lwahps2fpga`, un mapping de la mémoire physique est accessible par le fichier `/dev/mem`. Le programme `devmem` disponible sur la carte permet de faire des lectures et écriture via la ligne de commande. Un design de base est fourni et chargé automatiquement par u-boot au démarrage de la carte.

- Utilisez `devmem` pour lire l'état des boutons poussoirs et des switches. Les boutons sont mappés sur le `lwahps2fpga` bridge à l'offset `0x4B004` et les switches à l'offset `0x4B000`.

Test du design AXI4-Lite

Cette dernière étape permet de valider le fonction du design fait lors du laboratoire précédent depuis Linux.

- Reprenez votre programme de test du laboratoire précédent et adapter le pour en faire un application user-space utilisant `/dev/mem` qui permet de faire les mêmes tests depuis Linux.
- Compilez-le
- Copier le, soit :
 - À l'aide de `scp`
 - Comme en DRV, avec le dossier partager via NFS dans `/export/drv` sur la VM
 - En le copiant directement sur la carte SD
- Reprogrammer la FPGA à l'aide du script Python `pgm_fpga.py` (comme lors des autres laboratoires)
- Tester votre programme

Documents à rendre

Vous devez rendre un rapport à l'issue de ce laboratoire contenant les explications sur les différentes étapes de la réalisation de votre système.

Ainsi que les sources des fichiers créés (device tree, ITS, driver user-space)

Les fichiers sont à rendre sur Cyberlearn.